Save

Debouncing

Save

Debouncing is used to run a function after a specific amount of time. We decide that time, and the function gets called only after that delay — not immediately.

But if the user triggers the function again before the delay is over, the previous timer gets reset. Then, the function will run after the new delay.

Save

Example :-

Let's assume that debounce time is 500 milliseconds.

The user types: h, e, l, l, o — with a 200ms gap between each keypress.

On every keypress, the timer gets reset.

After typing o, the user stops typing...

Now, if the user doesn't type anything for the next 500ms, only then the function will finally run.

Save

Steps to Apply Debouncing :-

Step 1:- Create a function that convert your regular function into debounce function.

```
function debounce(fn, delay) {
  let timeout;

  return function (...args) {
    // Clear Previous Timer
    clearTimeout(timeout);

    // Set New Timer
    timeout = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}
```

Save

Step 2:- Pass your regular function into debounce function and convert your normal function into debounce function and use it.

```
function searchData() {
  console.log("Searching...");
}

const searchInput = document.getElementById("searchBox");
const debouncedSearchData = debounce(searchData, 1000);

searchInput.addEventListener("input", debouncedSearchData);
```

Save

ADVANTAGE

Debouncing helps reduce unnecessary function calls when a user types or scrolls quickly. It stops the function from running too often and waits until the user pauses. This prevents too many API calls and makes the website faster. As a result, it improves performance and gives a better user experience.

Save

Save this pdf for reference

Do you like this post

let me know your thoughts